

ASSESSMENT TOOL I – RELATIONSHIP VISUALIZATION LAB

Course : Data Handling and Visualization

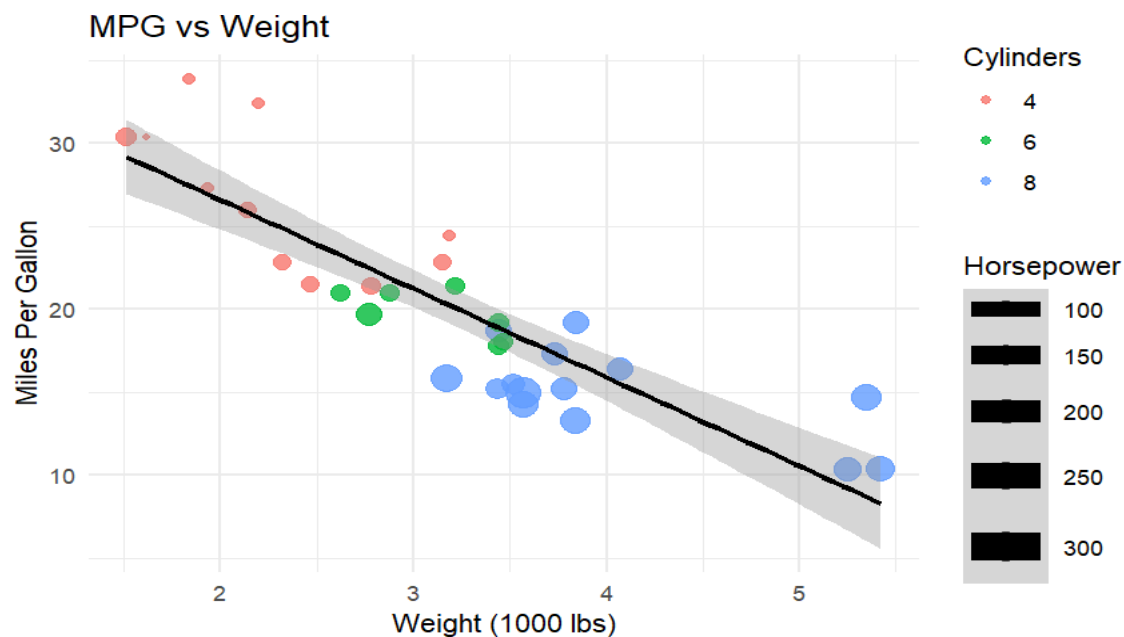
Name: C Nithisha

Reg. No: 192424375

QUESTION-1 Scatterplot with Regression Diagnostics

Aim

To visualize the relationship between miles per gallon (`mpg`) and car weight (`wt`), while also representing the number of cylinders and horsepower.



CODE:

```
library(ggplot2)
# Load built-in dataset
data(mtcars)
# Convert cyl to factor
mtcars$cyl <- as.factor(mtcars$cyl)
# Scatterplot
```

```
ggplot(mtcars, aes(x = wt, y = mpg,  
  color = cyl, size = hp)) +
```

```

geom_point(alpha = 0.8) +
geom_smooth(method = "lm", se = TRUE,
            color = "black") +
labs(
  title = "MPG vs Weight",
  x = "Weight (1000 lbs)",
  y = "Miles Per Gallon",
  color = "Cylinders",
  size = "Horsepower"
) +
theme_minimal()
# Pearson correlation
cor(mtcarsmpg, mtcarswt, method = "pearson")

```

Expected Result

The Pearson correlation coefficient is approximately:

-0.8677

Interpretation

The correlation between mpg and wt is approximately -0.868, indicating a strong negative relationship. As the weight of a car increases, its fuel efficiency in miles per gallon generally decreases. The regression line in the scatterplot also shows this downward trend.

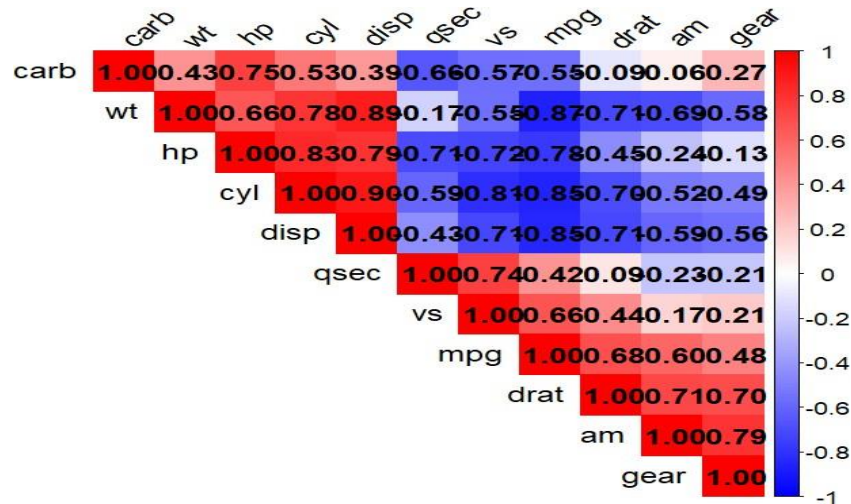
Rubric Connection

- **Relationship Visualization:** Scatterplot clearly shows `wt` vs `mpg`.
- **Relationship Analysis:** Pearson correlation measures the strength and direction.
- **Visualization Techniques:** Color represents `cyl`, size represents `hp`, and `geom_smooth()` adds regression diagnostics.
- **Clarity:** Labels, title, and legend make the graph easy to understand.

Question 2: Correlogram Construction

Aim

To calculate and visualize the correlations among all numeric variables in the `mtcars` dataset.



CODE:

```
library(corrplot)

# Load built-in dataset

data(mtcars)

# Correlation matrix

cor_matrix <- cor(mtcars)

# Display correlation matrix

print(cor_matrix)

# Correlogram

corrplot(

cor_matrix,

method = "color",

type = "upper",
```

```
order = "hclust",

addCoef.col = "black",

tl.col = "black",

tl.srt = 45,

col = colorRampPalette(

  c("blue", "white", "red") )(200))
```

Expected Important Results

For the standard `mtcars` dataset:

Relationship	Variables	Approximate Correlation
Strongest Positive	<code>disp</code> and <code>cyl</code>	0.902
Strongest Negative	<code>mpg</code> and <code>wt</code>	-0.868

Depending on whether duplicated upper/lower matrix pairs are counted, the same relationship may appear twice.

Interpretation

The strongest positive relationship is generally between **engine displacement (`disp`)** and **number of cylinders (`cyl`)**, meaning cars with more cylinders tend to have larger engines. The strongest negative relationship is between **`mpg` and `wt`**, showing that heavier cars generally have lower fuel efficiency.

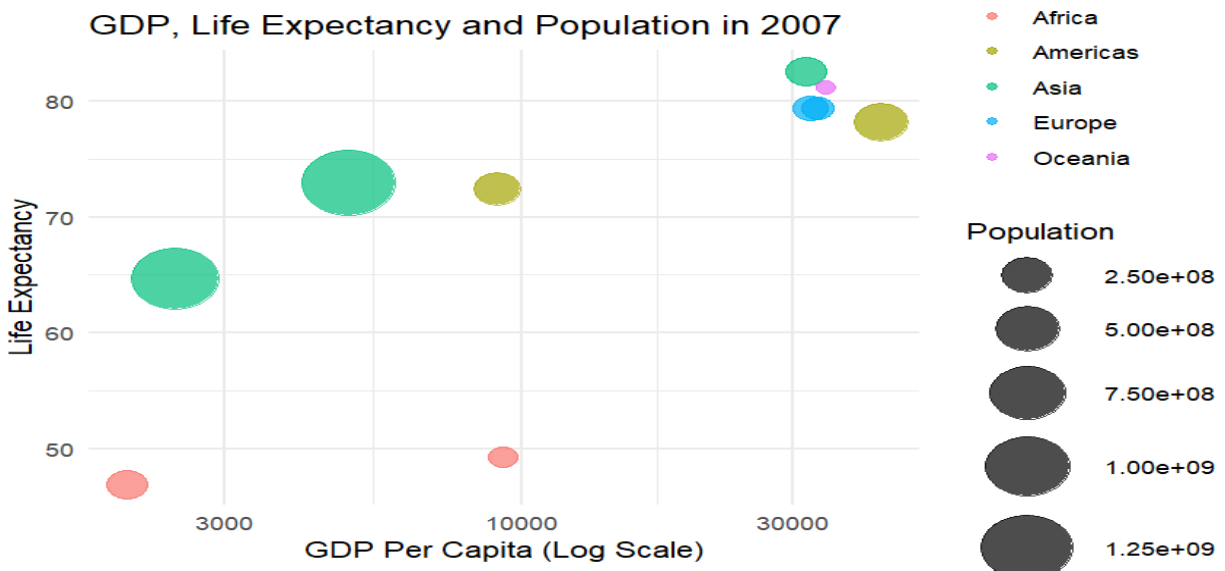
Rubric Connection

- **Relationship Visualization:** The correlogram displays relationships among all numeric variables.
- **Relationship Analysis:** Correlation coefficients quantify the strength and direction.
- **Visualization Technique:** Hierarchical clustering groups related variables together.
- **Accuracy:** Numeric labels allow exact interpretation of correlations.

Question 4: Bubble Chart for Multivariate Paired Data

Aim

To study the relationship between GDP per capita, life expectancy, population, and continent for countries in the year 2007.



CODE:

```
library(ggplot2)

# Create Gapminder-style dataset

gapminder_data <- data.frame(

  country = c(

    "India", "China", "USA", "Japan",

    "Germany", "Brazil", "Nigeria",

    "South Africa", "Australia", "UK"),

  continent = c(

    "Asia", "Asia", "Americas", "Asia",
```

```
"Europe", "Americas", "Africa",  
"Africa", "Oceania", "Europe" ),
```

```
year = c(  
  2007, 2007, 2007, 2007, 2007,  
  2007, 2007, 2007, 2007, 2007),
```

```
gdpPercap = c(  
  2452, 4959, 42952, 31656,  
  32170, 9065, 2013, 9269,  
  34435, 33203 ),
```

```
lifeExp = c(  
  64.7, 73.0, 78.2, 82.6,  
  79.4, 72.4, 46.9, 49.3,  
  81.2, 79.4),
```

```
pop = c(  
  1110396331,  
  1318683096,  
  301139947,  
  127467972,  
  82400996,  
  190010647,  
  135031164,  
  43997828,  
  20434176,  
  60776238 ))
```

```

# Filter year 2007

data_2007 <- subset(gapminder_data, year == 2007)

# Bubble chart

ggplot(
  data_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    size = pop,
    color = continent)) +
  geom_point(alpha = 0.7) +
  scale_x_log10() +
  scale_size_continuous(range = c(3, 15)) +
  labs(
    title = "GDP, Life Expectancy and Population in 2007",
    x = "GDP Per Capita (Log Scale)",
    y = "Life Expectancy",
    size = "Population",
    color = "Continent" ) +
  theme_minimal()

```

Interpretation

A visible pattern is that countries with higher GDP per capita generally have higher life expectancy. Many African countries form a cluster at lower GDP and lower life expectancy, while countries in Europe and Oceania are generally concentrated at higher GDP and higher life expectancy. A major outlier is represented by a very large bubble such as **China or India**, because of their extremely large populations.

Why Log Transformation?

GDP per capita values vary greatly between countries. Using:

`scale_x_log10()`

compresses the large range of values and makes the relationship between countries easier to observe.

Rubric Connection

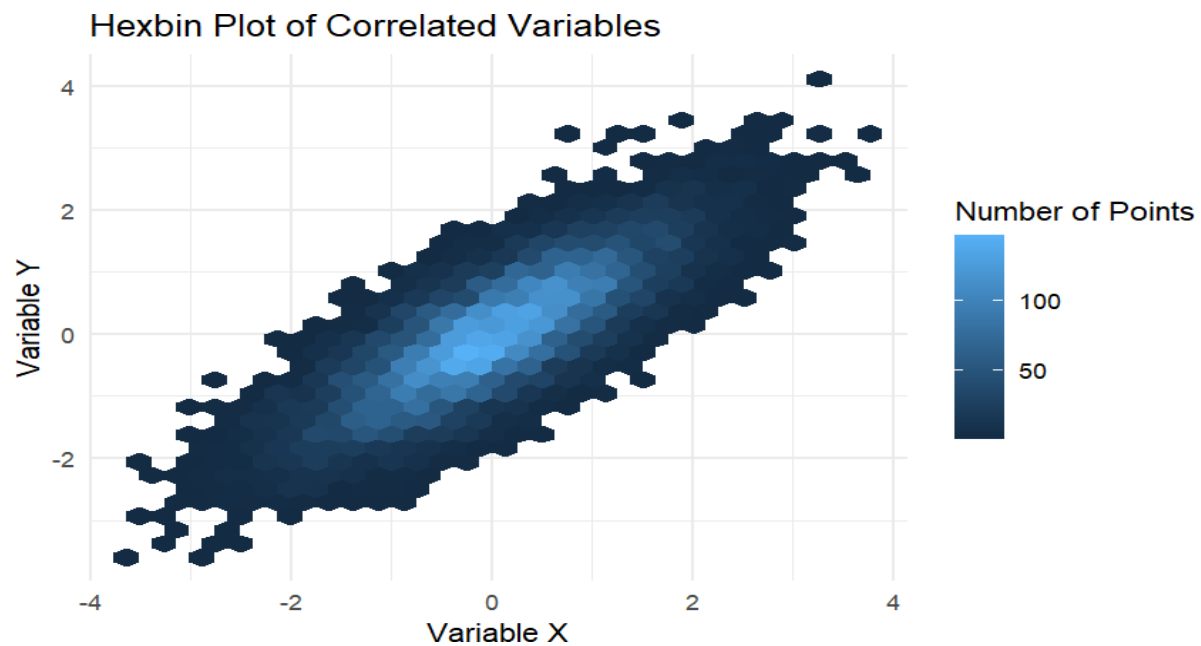
- **Relationship Visualization:** Displays four variables in one graph.
- **Relationship Analysis:** Shows the GDP–life expectancy relationship.
- **Visualization Techniques:** Uses bubble size, color encoding, and logarithmic transformation.
- **Clarity:** Axis labels and legends explain every variable.

Question 5: Hexbin Plot vs. Network Graph

Part A — Hexbin Plot

Aim

To visualize the relationship between two correlated variables containing **10,000 observations**.



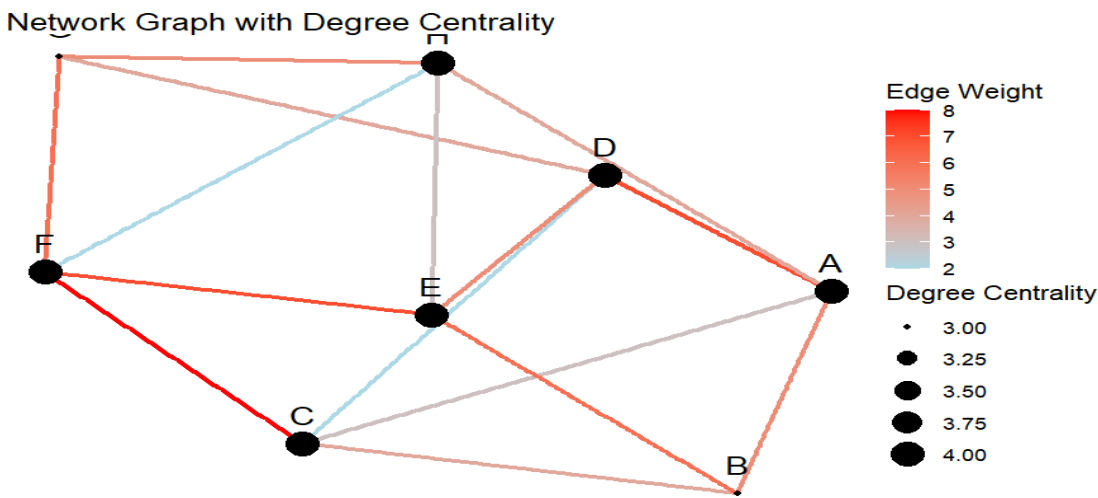
Interpretation

The plot shows a positive relationship between X and Y. A hexbin plot is preferred because 10,000 observations would cause significant overplotting in a normal scatterplot. Hexagonal bins summarize dense regions clearly.

Why Hexbin is Better Here

Standard Scatterplot	Hexbin Plot
Many points overlap	Groups points into hexagons
Difficult to identify dense regions	Clearly shows density
Overplotting occurs	Reduces overplotting
Less useful for very large datasets	Suitable for large datasets

Question 5(B): Network Graph



CODE:

```
# Q5B - Network Graph

library(igraph)

library(gggraph)

library(ggplot2)

# Create edge-list dataset

edges <- data.frame(

  from = c("A","A","A","B","B","C","C","D",

           "D","E","E","F","F","G","H"),

  to = c("B","C","D","C","E","D","F","E",

         "G","F","H","G","H","H","A"),

  weight = c(5,3,7,4,6,2,8,5,4,7,3,6,2,5,4))

# Display edge data

print(edges)

# Create undirected graph

g <- graph_from_data_frame(edges, directed = FALSE)

# Calculate degree centrality

degree_values <- degree(g)

# Add degree centrality to nodes

V(g)$degree <- degree_values

# Find node with highest centrality

highest_node <- names(which.max(degree_values))

cat("Degree Centrality:\n")

print(degree_values)
```

```

cat("\nNode with Highest Centrality:",
    highest_node, "\n")
# Create network graph
ggraph(g, layout = "fr") +
  geom_edge_link(
    aes(color = weight),
    width = 1 ) +
  geom_node_point(
    aes(size = degree) ) +
  geom_node_text(
    aes(label = name),
    vjust = -1,
    size = 5) +
  scale_edge_color_gradient(
    low = "lightblue",
    high = "red" ) +
  labs(
    title = "Network Graph with Degree Centrality",
    edge_color = "Edge Weight",
    size = "Degree Centrality") +
  theme_void()

```

Interpretation

The node printed by:

`highest_node`

is the node with the **highest degree centrality**. This means that the node has the largest number of direct connections with other nodes in the network. Therefore, it can be considered an important **hub node**, as it may play a major role in connecting different parts of the network and facilitating interaction or information flow.

Important: Since your actual `network_edges.csv` file is not included here, the exact highest-centrality node cannot be stated without seeing the edge list. Run the above code, and R will automatically print the correct node.